
Azure Hub-and-Spoke Project

Easy Learning + Deployment Guide

Build it. Test it. Understand why it works.

What you will build

A private Azure hub-and-spoke network with a P2S VPN, Azure Firewall, a Windows VMSS running an IIS/.NET app, a Linux VM running Flask, and Microsoft Entra ID sign-in for both application stacks.

Designed for

A learner who wants to repeat the project from scratch without reading a wall of text. Each section follows the same pattern: Goal -> Do -> Check -> Fix.

Project boundary

No on-premises environment, no DR region, no Bastion, no management/shared-services subnet, and no private endpoints.

Central India | Final lab version | 27 September 2026

Start Here: The Project in 60 Seconds

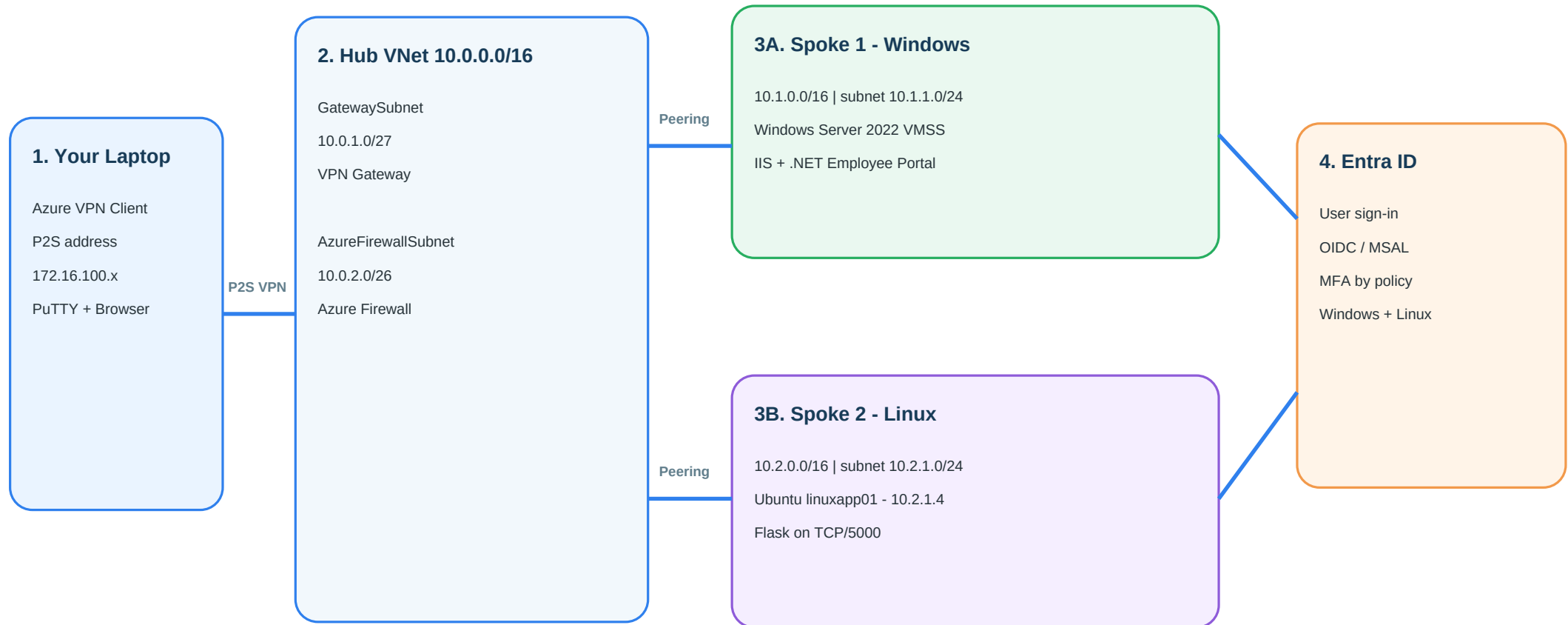
The simple idea

Your laptop first joins Azure through a Point-to-Site VPN. The Hub is the central network. The Hub is peered to two Spokes. Spoke 1 contains the Windows application. Spoke 2 contains the Linux application. Entra ID handles user sign-in.

Learning path

1	Build the network	Hub VNet, two special Hub subnets, two Spoke VNets and peering.
2	Connect your laptop	P2S VPN gives your laptop a private Azure route.
3	Secure the traffic	Azure Firewall, route tables and NSGs control allowed paths.
4	Deploy Windows app	Windows Server 2022 VMSS + IIS + ASP.NET Core/.NET.
5	Add Entra sign-in	Protect the Windows Employee Portal.
6	Deploy Linux app	Ubuntu VM + Python Flask + systemd.
7	Add Linux Entra sign-in	MSAL authorization-code flow.
8	Test and clean up	Prove every layer works, then destroy the lab to stop cost.

Architecture: Follow the Traffic



Read it left to right: Laptop -> VPN -> Hub -> Spoke -> Application -> Entra sign-in.

Not part of this project: on-premises, DR, Bastion, management subnet, private endpoints.

1. Know the Address Plan Before You Build

Rule to remember

Every VNet and subnet must have a unique, non-overlapping address range. Think of each VNet as a different neighborhood.

Area	CIDR	What lives here
Hub VNet	10.0.0.0/16	Central network
GatewaySubnet	10.0.1.0/27	VPN Gateway only
AzureFirewallSubnet	10.0.2.0/26	Azure Firewall only
Spoke 1 VNet	10.1.0.0/16	Windows workload
vmss-subnet	10.1.1.0/24	Windows VMSS
Spoke 2 VNet	10.2.0.0/16	Linux workload
linux-app-subnet	10.2.1.0/24	Ubuntu VM
P2S client pool	172.16.100.0/24	VPN-connected laptops

Checkpoint

If you can explain why 10.0.x.x is Hub, 10.1.x.x is Windows, and 10.2.x.x is Linux, you already understand the most important part of the topology.

2. Services Explained in Plain English

Hub VNet

The central network. Shared connectivity lives here.

GatewaySubnet

A reserved subnet used by Azure VPN Gateway. Do not place normal VMs here.

VPN Gateway

The door your laptop uses to enter the private Azure network.

AzureFirewallSubnet

A reserved subnet for Azure Firewall.

Azure Firewall

A central traffic-control point. It can allow, deny and log traffic.

VNet Peering

A private connection between VNets over the Azure backbone.

NSG

A subnet/NIC-level rule list that allows or blocks ports and sources.

UDR / Route Table

Tells Azure where traffic should go next, such as through the firewall.

VM Scale Set

A group of similar VMs that can scale out or in.

IIS

Microsoft web server used to host the Windows .NET application.

Flask

A lightweight Python web framework used by the Linux application.

Microsoft Entra ID

The identity provider used for sign-in. MFA is enforced when your Entra policy requires it.

3. Prepare Your Laptop

Install or have access to these tools:

1. Azure CLI - sign in and verify the subscription.
2. Terraform - deploy the Azure infrastructure.
3. Azure VPN Client - connect to the Hub privately.
4. PuTTY and PuTTYgen - SSH to the Linux VM.
5. .NET SDK - build/publish the Windows Employee Portal.

Quick check

```
az --version
az login
az account show
terraform version
```

Working folder

D:\Rajan\Cloud Project\Azure Hub-Spoke Architecture\terraform

Safety rule

Never put Terraform state, PFX files, SSH private keys, .env files or client secrets in a public repository.

4. Terraform: Use the Same 5-Command Rhythm

Think of Terraform like this

init = prepare, validate = check, plan = preview, apply = build, destroy = remove.

1. Open PowerShell in the Terraform project folder.

2. Initialize providers.

```
terraform init
```

3. Format and validate the configuration.

```
terraform fmt -recursive  
terraform validate
```

4. Preview exactly what Azure will change.

```
terraform plan -out tfplan
```

5. Build it.

```
terraform apply tfplan
```

Useful after deployment

```
terraform output  
terraform state list
```

Expected result

Terraform finishes without errors and the Azure resources appear in the intended resource group.

5. Build the Network First

Do not start with the applications. First create the roads they will use.

1. Create Hub VNet: 10.0.0.0/16.
2. Inside Hub, create GatewaySubnet: 10.0.1.0/27.
3. Inside Hub, create AzureFirewallSubnet: 10.0.2.0/26.
4. Create Spoke 1 VNet: 10.1.0.0/16 and vmss-subnet: 10.1.1.0/24.
5. Create Spoke 2 VNet: 10.2.0.0/16 and linux-app-subnet: 10.2.1.0/24.
6. Create Hub <-> Spoke 1 peering.
7. Create Hub <-> Spoke 2 peering.

Checkpoint

The VNets should be able to exchange private traffic through peering. Later, gateway transit lets the spokes use the Hub VPN gateway.

6. P2S VPN: Give Your Laptop a Private Path

Goal

When connected, your laptop receives an address from 172.16.100.0/24 and can reach the private spoke networks.

Create the certificate pair in PowerShell

```
$root = New-SelfSignedCertificate `
  -Type Custom -KeySpec Signature `
  -Subject "CN=HubSpokeP2SRoot" `
  -KeyExportPolicy Exportable -HashAlgorithm sha256 `
  -KeyLength 2048 -CertStoreLocation "Cert:\CurrentUser\My" `
  -KeyUsageProperty Sign -KeyUsage CertSign

$client = New-SelfSignedCertificate `
  -Type Custom -DnsName "HubSpokeP2SClient" `
  -KeySpec Signature -Subject "CN=HubSpokeP2SClient" `
  -KeyExportPolicy Exportable -HashAlgorithm sha256 `
  -KeyLength 2048 -CertStoreLocation "Cert:\CurrentUser\My" `
  -Signer $root
```

1. Upload the root public certificate to the P2S configuration.
2. Keep the client private certificate only on authorized machines.
3. Configure P2S pool: 172.16.100.0/24.
4. Download/import the Azure VPN Client profile.
5. Connect the VPN before testing private IPs.

7. VPN Test: Prove the Network Before Touching the App

From Windows PowerShell:

```
Test-NetConnection 10.2.1.4 -Port 22  
Test-NetConnection 10.2.1.4 -Port 5000
```

Your successful result

InterfaceAlias: Azure-Hub-Spoke-P2S
SourceAddress: 172.16.100.2
TcpTestSucceeded: True

What this proves

Your laptop entered Azure through P2S and successfully reached the Linux private IP in Spoke 2.

Small trap

Get-VpnConnection may not show an Azure VPN Client profile like a classic Windows phone-book VPN. Real connectivity tests are more useful here.

8. Firewall, NSG and Routing - Keep It Simple

Three layers, three jobs

Firewall: central inspection/control.

NSG: allow/deny at subnet or NIC level.

UDR: tells traffic which next hop to use.

Typical spoke route idea

Destination: 0.0.0.0/0

Next hop: Virtual appliance

Next hop IP: <Azure Firewall private IP>

Minimum useful rules for this lab

1. Linux SSH: TCP/22 only from trusted VPN/client sources.
2. Linux Flask: TCP/5000 only where required for the lab.
3. Windows/IIS: expose only the HTTP/HTTPS bindings you actually use.

Important

Do not force traffic through a firewall until the required firewall rules exist. A bad UDR can break package installation, Entra sign-in and application access.

9. Spoke 1: Windows VMSS + IIS + .NET

What lives here

Windows Server 2022 VM Scale Set in 10.1.1.0/24. IIS hosts the ASP.NET Core/.NET Employee Portal.

Application folder

C:\WebApps\EmployeePortal

Build the application

```
cd C:\WebApps\EmployeePortal
dotnet restore
dotnet build
dotnet list package
dotnet publish -c Release -o C:\WebApps\EmployeePortal-Publish
```

Packages used

Microsoft.Identity.Web 4.15.0

Microsoft.Identity.Web.UI 4.15.0

Your recorded build

0 warnings and 0 errors.

10. Put the .NET App Behind IIS

1. Install IIS on the Windows Server image/instance.
2. Install the ASP.NET Core Hosting Bundle that matches the runtime.
3. Create the IIS site/application pool.
4. Stop the app pool before replacing files.

```
Import-Module WebAdministration
Stop-WebAppPool -Name "EmployeePortal"

Remove-Item "C:\inetpub\EmployeePortal\*" -Recurse -Force
Copy-Item "C:\WebApps\EmployeePortal-Publish\*" `
  "C:\inetpub\EmployeePortal\" -Recurse -Force

Start-WebAppPool -Name "EmployeePortal"
iisreset
```

Why stop the app pool?

You hit a locked EmployeePortal.dll during deployment. IIS was using the file. Stopping the pool releases it so the new publish can be copied.

VMSS lesson

Do not configure only one scale-set instance by hand in a real deployment. Automate the setup so new instances are identical.

11. Protect the Windows App with Microsoft Entra ID

Authentication story

User opens the Employee Portal -> app redirects to Entra -> user signs in -> MFA occurs if required by policy -> Entra returns the user to /signin-oidc.

Entra app registration

1. Microsoft Entra ID -> App registrations -> New registration.
2. Use a Web application registration.
3. Register the exact redirect URI used by the app.

Local redirect used during development

`http://localhost:5257/signin-oidc`

appsettings.json pattern

```
"AzureAd": {  
  "Instance": "https://login.microsoftonline.com/",  
  "TenantId": "<TENANT-ID>",  
  "ClientId": "<CLIENT-ID>",  
  "CallbackPath": "/signin-oidc"  
}
```

12. Windows Entra Code Pattern

Program.cs core authentication setup:

```
using Microsoft.AspNetCore.Authentication.OpenIdConnect;
using Microsoft.Identity.Web;
using Microsoft.Identity.Web.UI;

builder.Services
    .AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApp(
        builder.Configuration.GetSection("AzureAd"));

builder.Services
    .AddControllersWithViews()
    .AddMicrosoftIdentityUI();

app.UseAuthentication();
app.UseAuthorization();
```

Error: AADSTS50011

Meaning: the redirect URI sent by the app does not exactly match the URI in Entra.

Fix: compare scheme, hostname, port and path character-for-character.

Error: AADSTS700054

Meaning: the authentication response/configuration did not match the app registration.

Fix: use the correct Web/OIDC authorization-code configuration rather than randomly enabling settings.

13. Spoke 2: First Connect to the Linux VM

Linux VM

Ubuntu 22.04.5 LTS | hostname linuxapp01 | private IP 10.2.1.4

Before SSH, prove port 22 is reachable

```
Test-NetConnection 10.2.1.4 -Port 22
```

If SSH says Permission denied (publickey)

1. Open PuTTYgen and generate a new RSA key pair.
2. Save both the public key and private .ppk securely.
3. Azure Portal -> VM -> Reset password or keys -> Add SSH public key.
4. Username: azureadmin. Paste the public key and update.
5. PuTTY -> SSH -> Auth/Credentials -> select the matching .ppk.
6. Connect to 10.2.1.4 on port 22 while VPN is connected.

Why this happened in your lab

Terraform referenced id_rsa.pub, but the matching original private key was not available on the laptop.

14. Inspect the Linux App Before Changing Anything

```
hostname
cat /etc/os-release
python3 --version
sudo ss -tulpn
curl http://localhost:5000
curl http://10.2.1.4:5000
```

What you found

Python 3.10.12
SSH listening on 0.0.0.0:22
Flask listening on 0.0.0.0:5000
Application path: /opt/flaskapp/app.py

Find the running process

```
ps -fp <PID>
sudo readlink -f /proc/<PID>/cwd
sudo tr '\0' ' ' < /proc/<PID>/cmdline
echo
```

Linux command reminder

Test-NetConnection is PowerShell, not Bash. On Linux use curl, ss, nc and similar Linux tools.

15. Create a Clean Python Virtual Environment

You first saw: ensurepip is not available. The fix was to install python3.10-venv.

```
cd /opt/flaskapp
sudo apt update
sudo apt install -y python3.10-venv
sudo rm -rf /opt/flaskapp/.venv
sudo python3 -m venv /opt/flaskapp/.venv
sudo chown -R azureadmin:azureadmin /opt/flaskapp/.venv
source /opt/flaskapp/.venv/bin/activate
```

Install the app packages

```
pip install --upgrade pip
pip install Flask msal python-dotenv requests
pip list | grep -Ei "Flask|msal|dotenv|requests"
```

Versions observed in your lab

Flask 3.1.3 | msal 1.39.0 | python-dotenv 1.2.3 | requests 2.34.2

16. Create the Linux Entra App Registration

1. Microsoft Entra ID -> App registrations -> New registration.
2. Name it something clear, such as Linux Employee Portal.
3. Use a single-tenant registration for this lab.
4. Authentication -> Add platform -> Web.
5. Add redirect URI: `http://localhost:5000/getAToken`.
6. Create a client secret for the lab and copy its VALUE immediately.

Security

A client secret was visible during the lab. Treat any exposed secret as compromised: revoke it and create a new one. This guide intentionally does not contain the real secret.

17. Store Linux Entra Settings in .env

```
TENANT_ID=<YOUR-TENANT-ID>
CLIENT_ID=<YOUR-CLIENT-ID>
CLIENT_SECRET=<YOUR-ROTATED-SECRET>
REDIRECT_PATH=/getAToken
```

Protect the file

```
sudo chown azureadmin:azureadmin /opt/flaskapp/.env
chmod 600 /opt/flaskapp/.env
```

Check that values exist without printing the secrets

```
python -c "from dotenv import load_dotenv; import os; load_dotenv(); print(
'Tenant:', bool(os.getenv('TENANT_ID'))),
'Client:', bool(os.getenv('CLIENT_ID'))),
'Secret:', bool(os.getenv('CLIENT_SECRET')))"
```

If nano says Permission denied

The file/folder is owned by another user. Fix ownership/permissions or edit it with sudo, then keep permissions restrictive.

18. Add Entra Login to Flask

Flow to remember

/login creates an MSAL authorization flow -> browser goes to Entra -> /getAToken receives the response -> user claims go into the Flask session -> /dashboard becomes available.

```
import os, secrets, msal
from flask import Flask, redirect, request, session, url_for
from dotenv import load_dotenv

load_dotenv("/opt/flaskapp/.env")
app = Flask(__name__)
app.secret_key = secrets.token_hex(32)

TENANT_ID = os.getenv("TENANT_ID")
CLIENT_ID = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
AUTHORITY = f"https://login.microsoftonline.com/{TENANT_ID}"
REDIRECT_URI = "http://localhost:5000/getAToken"

def build_msal_app():
    return msal.ConfidentialClientApplication(
        CLIENT_ID,
        authority=AUTHORITY,
        client_credential=CLIENT_SECRET)
```

19. Flask Login, Callback and Dashboard

```
@app.route("/login")
def login():
    flow = build_msal_app().initiate_auth_code_flow(
        scopes=[], redirect_uri=REDIRECT_URI)
    session["flow"] = flow
    return redirect(flow["auth_uri"])

@app.route("/getAToken")
def authorized():
    result = build_msal_app().acquire_token_by_auth_code_flow(
        session.get("flow", {}), request.args)
    if "error" in result:
        return result.get("error_description", "Authentication failed"), 400
    session["user"] = result.get("id_token_claims")
    return redirect(url_for("dashboard"))

@app.route("/dashboard")
def dashboard():
    user = session.get("user")
    if not user:
        return redirect(url_for("login"))
    return f"<h1>Employee Dashboard</h1><p>Welcome {user.get('name', 'Employee')}</p>"
```

Health endpoint

```
@app.route("/health")
def health():
    return {
        "status": "healthy",
        "server": socket.gethostname(),
        "spoke": "spoke-2"
    }
```

20. Run Flask Automatically with systemd

Why systemd?

You do not want the app to stop when your SSH/PuTTY window closes. systemd keeps it running as a Linux service.

`/etc/systemd/system/flaskapp.service`

```
[Unit]
Description=Azure Hub Spoke Flask Application
After=network.target

[Service]
User=root
WorkingDirectory=/opt/flaskapp
ExecStart=/opt/flaskapp/.venv/bin/python /opt/flaskapp/app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Apply and test

```
sudo systemctl daemon-reload
sudo systemctl restart flaskapp
sudo systemctl status flaskapp --no-pager
curl http://localhost:5000/health
```

Your successful result

flaskapp.service = active (running), using /opt/flaskapp/.venv/bin/python. The /health endpoint returned healthy.

21. Open the Linux App Safely from Your Laptop

Simple trick

Use an SSH local tunnel. Your browser opens localhost:5000, but PuTTY carries that traffic through SSH to the Linux VM.

PuTTY settings

1. Connection -> SSH -> Tunnels.
2. Source port: 5000.
3. Destination: 127.0.0.1:5000.
4. Select Local + Auto.
5. Click Add. You should see L5000 127.0.0.1:5000.

Then test on Windows

```
Test-NetConnection localhost -Port 5000
```

Open in your browser

```
http://localhost:5000
```

Full path

Browser -> localhost:5000 -> PuTTY tunnel -> SSH over P2S VPN -> linuxapp01:5000 -> Flask -> Entra sign-in -> callback.

22. Final Testing Checklist

Network tests

```
Test-NetConnection 10.2.1.4 -Port 22  
Test-NetConnection 10.2.1.4 -Port 5000
```

Linux tests

```
sudo ss -tulpn  
curl http://localhost:5000/health  
curl http://10.2.1.4:5000  
sudo journalctl -u flaskapp -n 50 --no-pager
```

Windows app tests

```
cd C:\WebApps\EmployeePortal  
dotnet build  
dotnet list package
```

Authentication success means

Unauthenticated user is redirected to Entra, sign-in completes, MFA appears when required by policy, the callback succeeds, and the protected page opens.

23. Troubleshooting Cheat Sheet

Problem	Meaning	Fast fix
AADSTS50011	Redirect mismatch	Match scheme + host + port + path exactly.
AADSTS700054	Auth response/config mismatch	Use correct Web/OIDC auth-code configuration.
DLL access denied	IIS has file open	Stop app pool before copying publish files.
Permission denied (publickey)	Wrong/missing private key	Add new public key; use matching .ppk.
venv creation failed	python3.10-venv missing	Install package, remove bad .venv, recreate.
Test-NetConnection not found	Running PowerShell command in Linux	Use it on Windows; use curl/ss/nc on Linux.
App dies after SSH closes	Not running as service	Use systemd.
MSAL missing after restart	systemd uses system Python	ExecStart must use .venv/bin/python.

24. What You Would Improve for Production

Linux web server

Flask's built-in development server is fine for this lab. For production, use Gunicorn behind Nginx or another production web tier.

Linux service identity

Run the application under a dedicated non-root service account instead of root.

Secrets

Prefer Azure Key Vault and managed identity where practical. Avoid long-lived client secrets.

HTTPS and DNS

Use a real DNS name, HTTPS and a valid certificate for a production application.

VMSS automation

Use an image, extension, cloud-init/PowerShell DSC, pipeline or another repeatable method so scale-out instances are identical.

Monitoring

Add Azure Monitor alerts and sensible Log Analytics retention for VPN, Firewall, compute and app health.

25. Finished Learning? Stop the Cost

Why cost rises

Always-on network services such as VPN Gateway and Azure Firewall can continue charging even when you are not actively using the lab. VMSS, the Linux VM, disks, public IPs and monitoring can also add cost.

Keep these before deleting

1. Terraform .tf files.
2. Application source code.
3. This learning guide and architecture evidence.
4. Screenshots that do not expose credentials.

Destroy the lab

```
cd "D:\Rajan\Cloud Project\Azure Hub-Spoke Architecture\terraform"
terraform plan -destroy
terraform destroy
```

After Terraform destroy

Check Azure Portal for anything created manually. Confirm the VPN Gateway, Azure Firewall, public IPs, VMSS, Linux VM/disks, route tables, NSGs and lab monitoring resources are gone if they were only for this project.

26. Rebuild From Scratch - One-Page Checklist

- [] **01** Sign in to Azure and select the correct subscription.
- [] **02** terraform init -> validate -> plan -> apply.
- [] **03** Create Hub VNet 10.0.0.0/16.
- [] **04** Create GatewaySubnet 10.0.1.0/27.
- [] **05** Create AzureFirewallSubnet 10.0.2.0/26.
- [] **06** Create Spoke 1 10.1.0.0/16 + vmss-subnet 10.1.1.0/24.
- [] **07** Create Spoke 2 10.2.0.0/16 + linux-app-subnet 10.2.1.0/24.
- [] **08** Create Hub/Spoke peerings and gateway transit.
- [] **09** Deploy Azure Firewall, NSGs and required routes.
- [] **10** Deploy VpnGw1AZ; P2S pool 172.16.100.0/24; certificate auth.
- [] **11** Connect Azure VPN Client and test private routes.
- [] **12** Deploy Windows Server 2022 VMSS and IIS/.NET Employee Portal.
- [] **13** Register Windows app in Entra and validate /signin-oidc.
- [] **14** Deploy Ubuntu 22.04.5 Linux VM.
- [] **15** Configure SSH key and connect over VPN.
- [] **16** Deploy Flask app in /opt/flaskapp.
- [] **17** Create .venv and install Flask/MSAL/python-dotenv/requests.
- [] **18** Register Linux Web app in Entra with localhost:5000/getAToken.
- [] **19** Secure .env and update Flask authentication code.
- [] **20** Run Flask through systemd using .venv Python.
- [] **21** Create PuTTY local tunnel and test browser sign-in.
- [] **22** Capture evidence and destroy resources when finished.

27. Quick Command Card

Terraform

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
terraform state list
terraform destroy
```

Windows / VPN

```
Test-NetConnection 10.2.1.4 -Port 22
Test-NetConnection 10.2.1.4 -Port 5000
route print
ipconfig /flushdns
```

Linux

```
hostname
cat /etc/os-release
sudo ss -tulpn
curl http://localhost:5000/health
sudo systemctl status flaskapp --no-pager
sudo journalctl -u flaskapp -n 50 --no-pager
```

.NET

```
dotnet restore
dotnet build
dotnet list package
dotnet publish -c Release -o C:\WebApps\EmployeePortal-Publish
```

You Finished a Real Multi-Stack Azure Lab

What you actually practiced

Networking

Hub-and-spoke VNets, subnet design, VNet peering, gateway transit, P2S VPN, routing, NSGs and Azure Firewall.

Windows workload

Windows Server VMSS, IIS, .NET publishing and Microsoft Entra authentication.

Linux workload

Ubuntu, SSH/PuTTY, Python virtual environments, Flask, systemd and MSAL authentication.

Operations

Connectivity testing, application health checks, troubleshooting, credential hygiene and cost teardown.

The goal is not to memorize every command. The goal is to understand the path: network first -> private access -> workload -> identity -> testing -> cleanup.